

2 DMA - Based Data Transfer Algorithm:-

START

Input: I/O device, memory Address, data size.

1. Initialize DMA controller.
2. Set DMA destination = memory Address.
3. Set DMA Source = I/O device.
4. Set transfer size = data size.
5. Configure DMA mode = transfer mode.
6. Enable DMA Interrupt.
7. Send DMA request to I/O device.
8. WHILE transfer is not complete.

DO

DMA controller transfers data.

CPU performs other tasks.

END WHILE

9. when transfer is complete:

DMA controller generates interrupt.

10. CPU receives DMA interrupt.
11. CPU executes DMA interrupt service routine.
12. Check transfer status.
13. Clear DMA interrupt flag.
14. If transfer successful then,
report "DMA transfer completed".

ELSE

report "DMA transfer Error".

END IF

STOP.

2. priority - Based vector interrupt handling :-

START

1. Initialize interrupt controller.
2. create priority levels for all devices.
3. Assign interrupt vector to each device.
4. WHILE system is running DO
IF interrupt request occurs.

THEN

Identify interrupting device
Determine priority of the interrupt.
Select highest - priority pending interrupt.
Save current cpu state.
get interrupt vector address.
jump to corresponding interrupt.

Service Routine (ISR)

Execute ISR

clear interrupt request.

restore cpu state.

return to interrupted program.

END IF

END WHILE

STOP

3. dynamic selection of memory - mapped I/O and I/O-mapped :-

START

Input: latency requirement, Bandwidth requirement.

1. read latency requirement.

3. IF latency requirement is HIGH
AND
bandwidth requirement is HIGH.

THEN

select memory-mapped I/O

Allocate device registers
inside the memory address
space

ELSE IF latency requirement is
low AND

bandwidth requirement
is low THEN

select I/O-mapped I/O

Allocate separate I/O
address space

ELSE

compare system requirements

If memory address access is preferred.

THEN

select memory-mapped I/O

ELSE

select I/O-mapped I/O

END IF

4. configure selected I/O mechanism.

5. start data communication.

stop.

4. Bus Arbitration for PCIe, USB and Thunderbolt:-

START

Input: Multiple devices.

1. Initialize bus controller.

2. register all connected devices.

3. WHILE communication is required.

DO

collect bus access requests.

FOR each requesting device

DO

Determine device
priority

Determine bandwidth.

requirement

determine urgency

END FOR

select device with highest arbitration.

priority

Grant bus access to

selected device

Allow selected device to transfer data

monitor transfer

If transfer is completed.

THEN

Release bus

ENDIF

Check for new requests.

END WHILE

5. manage interface - specific communication.

pci $\rightarrow$ high-speed device communication.

USB $\rightarrow$ peripheral communication.

Thunderbolt $\rightarrow$ high speed external devices.

stop.

5. Hybrid I/O using polling and DMA interrupts:-

START

Input: I/O device, data size.

1. detects I/O device.

2. determine device speed.

3. If device speed is low THEN
use polling

WHILE device is not ready.

DO

Check device status.

END WHILE

Transfer data using polling.

ELSE

use DMA

configure DMA controller.

set source address.

set destination address.

set transfer size.

WHILE DMA transfer is in progress.

DO

cpu performs other tasks.

END IF

4. report transfer completion

stop.